numbers, for instance, there are some edge cases that can produce long meaningless paths[15], like periodic or transcendent numbers, or results of certain floating points operations like 0.1+0.2, due to issue with double precision representation[16].

- Tries have memory overhead for nodes and references. As we have seen, some implementations require each node to store an array of $|\sum|$ edges, where $\sum$ is the alphabet used - even if the node has few or no children at all.

In summary, the advice could be: use tries when you have to frequently perform prefix searches (longestPrefix or keysWithPrefix); use hash tables when data is stored on slow supports like disk, or whenever memory locality is important. In all intermediate cases, profiling can help you take the best decision.

Tries offer extremely good performance for many string-based operations. Due to their structure, though, they are meant to store an array of children for each node. This can quickly become expensive: the total number of edges for a trie with n elements can swing anywhere between $|\sum|*n$ and $|\sum|*n*m$, where m is the average word length, depending on the degree of overlap of common prefixes.

We have seen that we can use associative arrays, dictionaries in particular, to implement nodes, only storing edges that are not-null. Of course, this solution comes at a cost: not only the cost to access each edge (that can be the